Progress Tracking and Schedule Risk Monitoring

F5 and F6 preliminary design ROUND2

Student Project Planning and Progress Tracking System | Group9 Xingmang

Original report: Zhang Yuyang (1240017498), version 1.0, 25 October 2026, 31 pages. Editorial ROUND2 integration prepared for Wei Mingrui with Codex assistance. DRAFT PENDING OWNER AND TEAM REVIEW.

This editable reconstruction preserves the original 13 sections, meaningful valid design content, six named operations, ten planned scenarios and both appendices. No original DOCX or editable diagram source was supplied; the new native DOCX and draw.io files are reviewable editorial reconstructions, not claims of original-author approval.

Design position	Round2 result
Modules	Seven business modules MON-M1..7 plus thin MON-M0 facade using shared SH-COORD rules.
Source ownership	F1/F2 source data; F3 links; F4 scheduling; F5 progress/Blocked; F6 optional warning composition.
Status and availability	Three saved statuses; independent Blocked overlay; one revision and explicit capability states.
Evidence boundary	Design documents only. No application implementation, executed tests, measured performance or customer approval.

The original PDF and its SHA-256 are retained in Sources. This report uses the verified revised URS as primary requirements evidence and the shared Round2 cross-module protocol as a consistent proposed design contract. Formal URS priorities remain unchanged.

Document control

Item	Detail
Document	03_Progress_Risk_Module_Design
Title	Progress Tracking and Schedule Risk Monitoring — Preliminary Software Design
Version	ROUND2 editorial reconstruction; targeted consistency repair 26 October 2026
Date	25 October 2026
Author	Original source: Zhang Yuyang (1240017498). Round 2 editorial integration prepared for Wei Mingrui with Codex assistance; owner approval not implied.
Scope	F5 Progress Tracking and F6 Schedule Risk Monitoring: module design, business rules, interfaces and conceptual dashboard
Primary reviewers	Zhang Weiyi (F1/F2 snapshot and status-save boundary); Wei Mingrui (F4 schedule-result contract); whole team for consistency
Status	DRAFT PENDING OWNER AND TEAM REVIEW. Design only; no application implementation or executed tests.

Source basis and limitations

The original report used repository navigation and fixture evidence. Round 2 checks the available revised URS (24 pages), revised requirements analysis (36 pages), Chapter 6 lecture (105 pages) and actual Proposal(1) (8 pages), with source hashes and provenance in the project review records. The URS is primary; proposal/repository descriptions do not override it.

The previous missing-source limitation is resolved by verified local files. Availability and byte equivalence do not establish customer approval. This editable ROUND2 report is reconstructed from the 31-page PDF because no original DOCX/draw.io sources were supplied; it preserves its sections, meaningful valid content, 10 scenarios and appendices while correcting affected contracts and figures. Editorial changes remain pending owner/team review.

Design position

The dashboard is a read-only projection. F1/F2 own source records, F3 owns links, F4 owns schedule facts, F5 owns Blocked/progress and F6 composes supported warnings. Shared SH-COORD applies the one access/snapshot/save protocol.

Contents

§	Section	Coverage
1	Introduction	Purpose, scope, audience and design status
2	Requirements review	FR/NFR/DR traceability and release priority
3	Design context and principles	Position in the architecture and non- negotiable rules
4	Module decomposition	Candidate modules, evaluation, refined hierarchy and responsibilities
5	Data and status model	Saved status, derived raw status with a separate Blocked overlay and invariants
6	Progress calculation design	Formula, edge cases and refresh after a saved change
7	Blocked task design	Dependency-derived blocks, reasons and manual updates
8	Risk monitoring design	Exactly three warning conditions and availability rules
9	Dashboard UI design	Information hierarchy, wireframe, states and interaction
10	Module interfaces	Contracts, data dependencies and error semantics
11	Validation scenarios	Ten proposed cases; none executed
12	Outstanding decisions	Questions that must be closed before implementation
13	Summary	Design outcome and review request
A	Appendix A — Traceability	Requirement → module → scenario
B	Appendix B — Glossary	Terms used in this document

How to review this document

If you want to review…	Read
Whether the right requirements are covered	Sections 2 and Appendix A, then the ten scenarios in Section 11
Whether the modules are well designed	Sections 4 and 5, especially cohesion/coupling decisions and invariants
Whether the calculations and blocked rule are correct	Sections 6 and 7, including 2 of 4 = 50% and the manual- update case
Whether the warnings stay within scope	Section 8, especially the availability gate and explicit exclusions
Whether the dashboard is understandable	Section 9 and Figure 2
Whether the interfaces are buildable	Section 10 and the open contracts in Section 12

Figures

Figure 1 (Section 4.4) is the module hierarchy and dependency view. Figure 2 (Section 9.3) is the low-fidelity dashboard wireframe. Separate editable .drawio files and full-resolution PNG files accompany this report.

1 Introduction

1.1 Purpose

This document defines the preliminary general design for F5 Progress Tracking and F6 Schedule Risk Monitoring in the Group 9 student project planning system. It converts the relevant requirements into a small set of cohesive modules, explicit data and calculation rules, preliminary input/output contracts, and a dashboard concept. It deliberately stops before implementation: there is no application code, database schema, API endpoint or automated test in this deliverable.

The central design question is how to show progress and risk without duplicating F1-F4 source/schedule logic. The answer remains a read-only monitoring subsystem consuming one authorized coherent ProjectSnapshot plus independently available F3/F4 views. Schedule-only failure does not fail reliable basic progress.

1.2 Scope

Dimension	Boundary
In scope	F5: three raw-status counts and a separate Blocked overlay, completion percentage, blocked-task reasons, assigned-task overview and refresh after a saved change. F6: unfinished overdue tasks, blocked schedule-critical tasks and calculated project-finish versus project-deadline conflict.
Out of scope	Task creation/editing, dependency editing, critical-path calculation, authentication implementation, persistence design, notifications, AI prediction, automatic rescheduling or workload balancing.
Design depth	Module responsibilities, dependency direction, data contracts, business rules, error and empty states, conceptual UI and validation scenarios.
Implementation status	None. All interface names are proposals for team review, not existing APIs.

1.3 Intended audience

The primary audience is the Group 9 design review: Zhang Weiyi for the F1/F2 snapshot and status-save boundary, Wei Mingrui for the F4 schedule-result boundary, and the course instructor for design quality. A secondary audience is the future implementer who needs enough precision to build the module without inventing rules.

1.4 Design status and honesty statement

This is a proposal, not a result

The document contains no claim that any test has passed, that any screen has been built, or that any performance target has been met. Section 11 lists proposed validation scenarios with expected results. They must be executed later and the actual results recorded separately.

2 Requirements review

2.1 How the requirements are used

The review distinguishes three things that are often mixed together: (a) the user-visible outcome required by the URS, (b) the data or service on which that outcome depends, and (c) the release priority when dependencies are incomplete. This separation matters because a schedule warning can be required by the URS yet still be impossible to display truthfully when F4 has no valid schedule result. The design must degrade by suppressing the warning, not by inventing a forecast.

2.2 Relevant functional requirements

ID	Required outcome (working statement)	Design response	Priority
FR-09	Display blocked tasks and the reasons they are blocked.	MON-M3 alone derives Blocked from valid F3 links and unfinished direct prerequisites. MON-M1 retains three raw-status counts.	Should Have
FR-13	Display a project progress dashboard.	Basic Dashboard: MON-M0 / MON-M1 / MON-M2 / MON-M4 / MON-M5 + UI. Blocked: MON-M3, dependent on Should Have FR-09/F3; D-01 pending.	Must Have
FR-14	Show task completion status.	MON-M1 counts saved statuses; MON-M2 computes completed/total. Blocked is a separate overlay.	Must Have
FR-15	Display schedule warnings with understandable reasons.	Optional warning composer reuses MON-M5 overdue facts and MON-M6/MON-M7 schedule-backed comparisons. Exactly three warning types.	Could Have
FR-04	Update Task Status	F2 authorizes and saves status, then SH-COORD requests refreshed F5 views without changing graph/duration.	Must Have; upstream
FR-05	Task Comments and Progress Notes	F2 owns permitted short notes. Notes explain work without changing task-count completion.	Unranked; upstream

Formal priority is not a delivery-order judgment: FR-09 Should; FR-13/14 Must; FR-15 Could. FR-13 basic overdue facts are Must without promoting the optional FR-15 warning layer. FR-13 Blocked display depends on Should-have F3/FR-09. If links are unavailable, retain reliable basic views and label blocker information unknown; full FR-13 acceptance and any F3 priority/scope adjustment require team/customer decision.

2.3 Non-functional and data constraints

The revised URS is now available and verified. The official names and meanings below replace the original draft's provisional labels. No acceptance threshold or customer approval is inferred from file availability.

ID	Quality lens used in this design	Concrete design response	How it is protected
NFR-01	Usability	One dashboard with a clear reading order: overall progress → status counts → personal tasks → blocked work → warnings. Plain language and text labels accompany colour.	UI review; scenario walkthroughs in Section 11
NFR-02	Browser and Device Support	Chrome/Edge desktop workflows; mobile viewing/simple status changes desirable. Exact versions and viewports pending.	Later agreed browser/device checks
NFR-03	Performance	O(T+E) projection. The 1-2 second expectation and typical15-40 tasks are not tested guarantees; conditions pending.	Single-pass counts; contract test on large fixture later
NFR-04	Security and Access Control	MG-M2 policy via SH-COORD authorizes project access before composition. Generic inaccessible-project errors return no private counts/names.	TC-07; backend integration test later

ID	Quality lens used in this design	Concrete design response	How it is protected
NFR-05	Reliability	Visible save errors, retained entered input, preserved valid saved data/plan and recovery after temporary unavailability.	Proposed save/reject/reopen/recovery scenarios, not executed
DR-04	Blocked Work as a Warning Condition	Retain raw task status, name unfinished direct prerequisites, and permit authorized manual updates.	TC-03/04/08; owner/customer transition policy pending

2.4 Must Have versus Could Have

Formal priority	Capability	Design dependency
Must Have	FR-13/14 basic progress, raw statuses, assignments, basic overdue facts and refresh	F1/F2 sources support MON-M0 / MON-M1 / MON-M2 / MON-M4 / MON-M5 + UI. Blocked separately uses MON-M3 and Should-have F3/FR-09; release decision D-01 remains pending.
Should Have	FR-09 Blocked labels and reasons; F3 dependency capability	F3 supplies links; F5 derives warning. Missing links are unknown, not no blockers.
Could Have	FR-15 entire warning layer, including overdue warning objects	Basic overdue list remains FR-13 Must Have; do not promote FR-15.
Could Have	FR-10/11/12 F4 analysis and refresh	Critical/finish warnings require current valid F4 results.
Excluded	AI forecasting, automatic reassignment, CPM in monitoring, persisted BLOCKED status	No unsupported feature or priority change is introduced.

2.5 Requirement-to-design traceability

Traceability remains bidirectional: FR-13 basic Dashboard → MON-M0 / MON-M1 / MON-M2 / MON-M4 / MON-M5 + UI. FR-13 Blocked subview → MON-M3; dependent on Should Have FR-09/F3; release decision D-01 remains pending. FR-09 → MON-M3 with MON-M1/MON-M4 views; FR-14 → MON-M1/MON-M2; FR-15 → MON-M5 facts reused by the optional composer and MON-M6/MON-M7. FR-04 is upstream status saving; FR-05 is the notes source.

3 Design context and principles

3.1 Position in the system

The monitoring subsystem sits between the saved project records, the schedule analysis result and the dashboard UI. F1/F2 own projects, members, tasks, assignments, deadlines and the three saved statuses. F3 owns valid prerequisite links and cycle rejection; F4 owns topological ordering, critical activities, project duration and estimated finish facts. F5/F6 read those results and add interpretation: what the numbers mean, which task is blocked, and why a warning exists.

Layer / area	Owns	Monitoring relationship
F1/F2 Project & Task Management	Project dates, members, task records, assignees, task deadlines, saved status, update authorization and persistence.	Upstream source. Monitoring calls a read-only snapshot port and never writes a task.
F3/F4 Dependency & Schedule Analysis	F3 owns valid dependency records; F4 owns schedule calculations and current result availability.	Two distinct read ports. F5 does not require F4 to derive Blocked.

Layer / area	Owns	Monitoring relationship
F5/F6 Monitoring (this design)	Raw-status counts, completion, direct Blocked explanations, assigned tasks, overdue facts and optional supported warnings.	Read-only projection. No persistence of derived values.
Frontend dashboard	Presentation, navigation, filter controls owned by the page, loading/empty/error states.	Consumes one DashboardSummary; does not recompute business rules.

3.2 Design principles

Single source of truth. Saved task status and dependencies remain authoritative. The dashboard does not maintain counters or a second status field.

Derived, not persisted. Blocked, completion percentage, overdue state and warnings are computed on read. None is stored as a new task status or a project column.

No duplicated schedule algorithm. Critical IDs and estimated project finish come from F4. Monitoring compares and explains; it does not run CPM or critical-path logic.

One consistent snapshot. Every summary is built from one project revision. Counts, blocked details, personal tasks and warnings cannot refer to different saves.

Explain before decorating. Every warning carries a reason and the evidence used. A red badge without a task, date or prerequisite name is not acceptable.

Fail closed on schedule claims. When the schedule result is unavailable, schedule-dependent warnings are suppressed. The UI may say that schedule analysis is unavailable, but it must not guess.

Read-only and side-effect free. Opening or refreshing the dashboard changes no task, date, dependency or assignment.

Explicit time. The as-of date is an input, not an ambient call to the system clock inside a calculation. This makes overdue tests reproducible.

3.3 Read-model boundary

Monitoring is a read model with a narrow purpose. It may compose data from several upstream records, but it does not become a repository. The boundary has three consequences:

The dashboard can be rebuilt at any time from saved inputs; it has no migration or backfill obligation for derived fields.

A bug in a calculation is fixed by changing the projection rule, not by repairing stored dashboard rows.

Access control must run before composition; otherwise a non-member could infer task names from an error or a partial response.

4 Module decomposition

4.1 Candidate modules from the brief

The brief proposes seven candidate modules under two functional areas. The first pass keeps their user-facing names but asks a stricter design question for each: what single reason for change does it have, what does it need, and what must it hide?

Area	Candidate module	Initial responsibility
Progress Tracking	Task Status Summary	Summarise tasks into raw statuses with separate overlays and counts.
Progress Tracking	Completion Percentage Calculation	Compute the project completion percentage from task status.
Progress Tracking	Blocked Task Identification	Find blocked tasks and name unfinished prerequisites.
Progress Tracking	Personal Assignment Overview	Show the current member's assigned tasks with context.
Risk Monitoring	Overdue Task Detection	Find unfinished tasks whose deadline has passed.

Area	Candidate module	Initial responsibility
Risk Monitoring	Critical Blocker Warning	Identify blocked tasks that are critical activities.
Risk Monitoring	Project Deadline Conflict Warning	Compare calculated project finish with the project deadline.

4.2 Evaluation criteria

Criterion	Question used in this review	Failure symptom
Modularity	Can the responsibility be described without 'and also'? Can it be tested alone?	A module mixes counting, dependency interpretation and UI formatting.
High cohesion	Do all parts change for the same reason and operate on the same concept?	One change forces edits in unrelated branches of the module.
Low coupling	Does it depend on a small, explicit contract rather than repositories, UI or ambient time?	Tests need a database or a rendered page to exercise a count.
Abstraction	Does the interface state the user-level intent rather than expose the algorithm?	UI calls a topological-sort helper to decide whether to show a warning.
Information hiding	Are raw records, intermediate maps and algorithm details kept out of callers?	The dashboard formats prerequisite arrays itself and can create a second interpretation.

4.3 Evaluation and refinement decisions

Candidate	Evaluation	Decision
Task Status Summary	Cohesive if limited to classification and counting. Mixing percentage into it would give two reasons to change: category rules and percentage/rounding policy.	Keep. Count the three stored statuses; Blocked is a separate overlay, not a mutually exclusive bucket.
Completion Percentage Calculation	Very high cohesion and low coupling: task statuses in, fraction/counts out. Does not need schedule data. Pure and easy to test for 2/4 = 50% and the empty case.	Keep as a pure function. Contract explicitly returns null for zero tasks.
Blocked Task Identification	Core derived rule with a distinct input (dependency edges). It must be shared by status counts, personal view and critical warning; duplicating it would create inconsistent reasons.	Keep. Compute once per snapshot and pass the result to consumers.
Personal Assignment Overview	It is a filtered projection, not a new source of task data. It needs blocked, critical and deadline context but should not recalculate them.	Keep as a view builder over already-derived context. Unassigned tasks are excluded from a member view but still included in project progress.
Overdue Task Detection	Independent of schedule analysis. One rule, one input date, one output list. High cohesion; strong boundary case around the deadline day.	Keep. Use an explicit as-of date; Completed is excluded.
Critical Blocker Warning	The intersection of two upstream facts: blocked state and the critical-task set. If merged into blocked detection, F5 would start to depend on F4 unnecessarily.	Keep separate. It is availability-gated and has no fallback critical-path calculation.
Project Deadline Conflict Warning	One project-level comparison, not a per-task warning. It needs a valid calculated finish, not a forecast. Mixing it with overdue tasks would confuse task lateness with project lateness.	Keep separate. Return at most one conflict warning.
Cross-cutting dashboard assembly	No candidate owns the ordering, access check, single-snapshot guarantee or one- response contract. Without it the UI would call six operations and risk inconsistent fragments.	Add a thin application coordinator, buildDashboardSummary, plus getScheduleWarnings as the warning composer. Do not add a separate class per formatter.

4 4 Refined module hierarchy

Seven business modules remain. MON-M1..5 provide F5 facts; optional F6 composes warnings using MON-M5 facts and MON-M6/MON-M7. The thin MON-M0 facade uses shared SH-COORD access/snapshot rules. Lines show decomposition, not runtime messages.

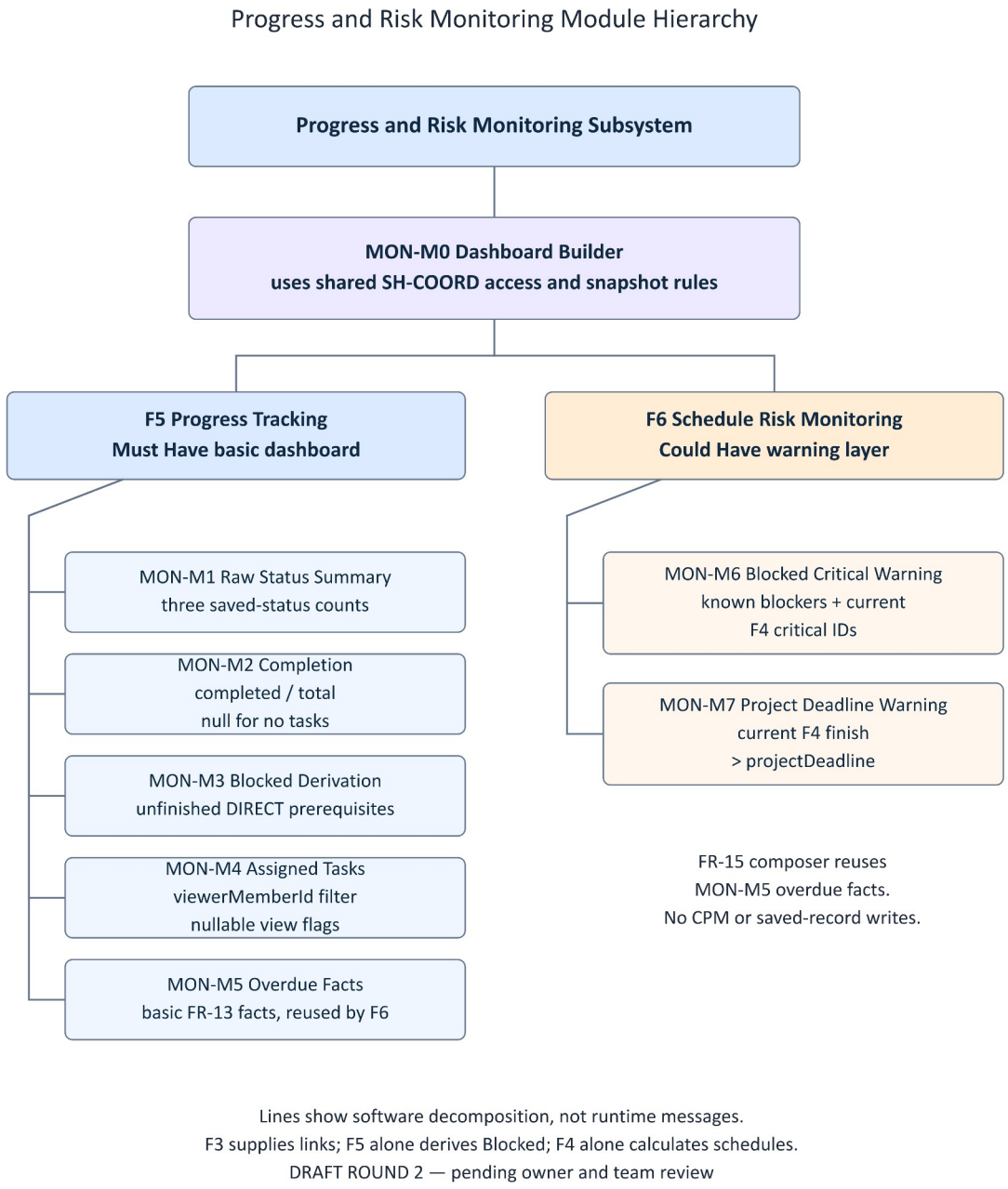


Figure1. Editable Round2 software hierarchy. The original valid functional subdivisions are retained; overdue facts are explicitly shared with the Must-have basic view. No CPM, source writing or extra service is added.

4.5 Module responsibilities

ID	Module	Responsibility	Explicitly not responsible for
MON-M0	Dashboard Builder	Use shared SH-COORD access/snapshot policy, compose reliable views at one revision, and return partial capability availability.	Persisting data; calculating CPM; rendering HTML; deciding page navigation.
MON-M1	Raw Status Summary	Count all tasks once by NOT_STARTED, IN_PROGRESS or COMPLETED; expose raw counts and IDs. Keep blockedCount separate.	Computing the percentage; naming prerequisites; filtering by viewer.
MON-M2	Completion	Compute Completed / distinct total and return an explicit empty-project result. Apply the agreed presentation precision at the boundary.	Counting Blocked as a separate subtraction; estimating effort or remaining hours.
MON-M3	Blocked Derivation	For each unfinished task, collect direct prerequisites that are not Completed; produce blocked indicator, names, raw statuses and a readable reason.	Persisting Blocked; locking status edits; recursively expanding root causes.
MON-M4	Assigned Tasks	Resolve viewerMemberId via authorized membership, then filter assigneeId. Preserve rawStatus and nullable Blocked/critical flags.	Changing project-wide counts; including unassigned tasks in a member's list.
MON-M5	Overdue Facts	Produce basic overdue facts for FR-13 independently of F4/F6; optional FR-15 reuses those facts.	Warning about completed work; forecasting a completion date; changing deadlines.
MON-M6	Blocked Critical Warning	Return warnings for blocked tasks whose IDs are in a valid critical-task set. Include direct blocker names.	Deriving the critical path; warning when schedule data is unavailable.
MON-M7	Project Deadline Warning	Compare the validated calculated project finish date with the project deadline; return one conflict with the date gap.	Compressing the plan; predicting delay; warning per task.

4.6 Cohesion, coupling and abstraction

The seven business modules are intentionally small. Each has one reason to change: a raw-status count rule (MON-M1), a formula or precision policy (MON-M2), a dependency interpretation (MON-M3), a personal filter (MON-M4), a date boundary (MON-M5), a criticality intersection (MON-M6), or a project-level date comparison (MON-M7). Warning composition is the getScheduleWarnings operation, not an eighth domain class; MON-M0 changes only when the use-case boundary or summary shape changes.

Coupling is kept low by passing values rather than repositories. MON-M1–MON-M7 do not know about HTTP, SQL, sessions, or the DOM. MON-M2 and MON-M5 are pure with respect to their inputs. MON-M3 is pure with respect to task and dependency lists. MON-M6 and MON-M7 depend on a small ScheduleResultView and an explicit availability flag. MON-M0 depends on ports, but only as a composition root.

The key abstraction remains saved facts versus displayed interpretations: Task.status is saved; TaskDisplayView exposes rawStatus and separate nullable Blocked/critical flags. F3 owns dependency edges; F5 alone derives Blocked. F4 calculates durations/critical IDs; F6 only interprets supplied facts.

Maintainability, testability and explainability remain useful design-quality goals. They are not the formal meaning of NFR-05; that requirement is Reliability.

5 Data and status model

5 1 Saved status and independent overlays

Only NOT_STARTED, IN_PROGRESS and COMPLETED are saved by F1/F2 in this proposed design. Human labels are presentation mappings. BLOCKED derives from unfinished direct prerequisites and never overwrites a stored status. Final transition and completion-before-prerequisite policies remain pending.

Concept	Values	Owner and meaning
TaskStatus	NOT_STARTED | IN_PROGRESS | COMPLETED	F1/F2 persisted source status
TaskDisplayView	rawStatus; isBlocked:boolean|null; isCritical:boolean|null	F5 view. Null means unevaluated; separate overlay, not counting category.
BlockedResult	availability; sourceRevision; known IDs/details or null	F5 derives once from current valid F3 links.
ScheduleAvailability	VALID | NO_TASKS | NOT_ENABLED | UNAVAILABLE	F4 result capability; reasonCode distinguishes failure/staleness.

5 2 Raw status and Blocked derivation

```
rawStatusCounts = count tasks by saved TaskStatus
// COMPLETED + IN_PROGRESS + NOT_STARTED = totalTasks
if dependencyView is NOT_ENABLED or UNAVAILABLE:
    isBlocked = null; blockedTaskIds = null
else for each task:
    isBlocked = task.status != COMPLETED
        AND any DIRECT prerequisite.status != COMPLETED
// blockedCount is a separate overlay; never add it to totalTasks.
```

Completed tasks are never unfinished blocked work. A raw IN_PROGRESS task may carry Blocked while still counted as IN_PROGRESS. A raw NOT_STARTED task remains in that count even when it also appears in the blocker panel. Authorized manual updates stay available; F5 does not create a force/lock workflow.

5 3 Worked classification

Task	Saved status	Direct prerequisites	Overlay / reason
A Requirements	COMPLETED	None	Not blocked
B Frontend	COMPLETED	A completed	Not blocked
C Backend	IN_PROGRESS	A completed	Not blocked
D Testing	NOT_STARTED	B completed; C in progress	Blocked by C only

Raw counts: COMPLETED2, IN_PROGRESS1, NOT_STARTED1; total4. blockedCount1 is independent. Completion2/4=50%. Source revision and dependency availability accompany every result.

5 4 Data invariants

A coherent authorized snapshot has unique stable task IDs and valid enum values. Conflicting duplicates are a source-integrity failure, not an arbitrary map overwrite.

Every task contributes once to one of the three raw-status counts. Their sum equals totalTasks. Blocked overlays do not change the completion numerator/denominator.

A completed task is absent from blockedTaskIds. Each known blocker reason names every incomplete direct prerequisite and no transitive root-cause invention.

Known empty links from enabled F3 mean no prerequisites; missing F3 data means unknown. No missing critical set or finish date is interpreted as no risk.

Derived results carry sourceRevision. SH-COORD authorizes one project/revision context for tasks, memberships and complete F3 edges; DependencyResultView.sourceRevision must equal snapshot.revision. After unrelated notes/roles change, re-read coherently or prove the validated complete edges and task universe unchanged before republishing at the current revision. Relevant task/edge changes require fresh validation. Without proof, dependency data is UNAVAILABLE/STALE_RESULT with edges=null; missing complete input uses DEPENDENCY_INPUT_UNAVAILABLE. These are design constraints, not implemented locks or transactions.

5 5 ScheduleResultView boundary

F3 owns valid links; F4 alone owns scheduling. Monitoring consumes the following small read-only result, not mutable CPM arrays or a second date formula.

Field	Contract
availability	VALID | NO_TASKS | NOT_ENABLED | UNAVAILABLE
sourceRevision	Same project revision as the coherent current snapshot; dependency input must also be proved current (Section 5.4).
durationDays; timings	Known for a nonempty VALID plan; null otherwise. Timing records have taskId, ES, EF, LS, LF, totalFloat.
criticalTaskIds	Full NumericPolicy-certified TaskId[] only for VALID current F4 analysis; null when unevaluated. MON never infers criticality from displayed rounded float or a local epsilon.
estimatedProjectFinish	Supplied only by F4 under selected D-04 candidate/start: plannedStartDate+ceil(T) for canonical unrounded resolved T; null when the start or mapping is unavailable.
reasonCode	NOT_ENABLED: FEATURE_NOT_ENABLED. UNAVAILABLE: INVALID_INPUT, CALCULATION_FAILED, STALE_RESULT or DEPENDENCY_INPUT_UNAVAILABLE. No failure reasonCode for VALID/NO_TASKS.
finishReasonCode	MISSING_PLANNED_START or DATE_MAPPING_UNRESOLVED explains null absolute finish while reliable relative analysis may remain VALID. D-04 remains PENDING.
diagnostics	Authorized INVALID_INPUT only: DurationInputDiagnostic[]={taskId:TaskId, field:"durationDays", issue:MISSING|NON_NUMERIC|NON_FINITE|NON_POSITIVE, message:string}. Identify detectable invalid task/field inputs; schedule facts stay null. Other states use diagnostics=null. Access denial returns generic PublicError without diagnostics; this does not decide F2 save policy.

Apply F4 gates in order: off → NOT_ENABLED/FEATURE_NOT_ENABLED; enabled empty tasks → NO_TASKS with null schedule payload; enabled nonempty tasks without a complete, valid, same-revision dependency view → UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE. Only then calculate. INVALID_INPUT, CALCULATION_FAILED and STALE_RESULT retain their distinct schedule meanings. edges=null is unknown; valid edges=[] is known no prerequisites. Notices describe capability and never invent delay warnings.

6 Progress calculation design

6 1 Basic rule

Completion uses task count, not duration, effort or elapsed hours. For a coherent nonempty snapshot, completedTasks/totalTasks*100 gives the percentage. Two completed tasks out of four give50%. Blocked work remains unfinished and contributes once to total, with no extra subtraction. Progress is project-wide, independent of the personal assignment filter.

6 2 Calculation algorithm and result

```
calculateProgress(tasks, sourceRevision):
    require coherent unique task IDs and valid TaskStatus
    total = count(tasks)
    completed = count(status == COMPLETED)
    if total == 0:
        return {availability:NO_TASKS, sourceRevision:sourceRevision,
                totalTasks:0, completedTasks:0, percentage:null}
    return {availability:VALID, sourceRevision:sourceRevision,
            totalTasks:total, completedTasks:completed,
            percentage:completed / total * 100}
```

Keep the exact numeric fraction for calculation. Percentage precision is a presentation policy: the original proposal of one decimal place with trailing .0 removed remains pending. Do not use rounded display values in later calculations. No tasks is a neutral no-work state; wording such as No tasks yet is proposed, not customer-approved.

6 3 Edge cases

Case	Expected result	Reason
Empty project	NO_TASKS; percentage null; no division	No denominator and no fabricated0%/0-day delivery.
NOT_STARTED	Included in total and NOT_STARTED count, even if blocked	Overlay does not erase raw status.
IN_PROGRESS	Included in total and IN_PROGRESS count, even if blocked	No invented fractional completion.
Blocked	Still unfinished; separate known overlay only	Not an extra task or fourth status.
COMPLETED with unfinished prerequisite	Completed count; isBlocked false	Completion-before-prerequisite workflow remains open; no F5 lock.
Unassigned	Included in project totals; not in a specific member list	No unsupported unassigned-risk warning.
Duplicate conflicting task ID	Reject unsafe source read; no arbitrary last-write-wins count	Snapshot-integrity boundary.
F3 unavailable	Progress/raw counts remain; blocker flags unknown	Missing links are not valid empty links.
F4 unavailable	Progress and basic overdue facts remain	No CPM dependency for completion.

6 4 Worked example

Task	Saved status	Numerator	Denominator
A	COMPLETED	Yes	Yes
B	COMPLETED	Yes	Yes
C	IN_PROGRESS	No	Yes
D, blocked by C	NOT_STARTED	No	Yes

Result2/4=50%. Raw counts2+1+1=4; blockedCount1 is a separate overlay, not an extra bucket.

6 5 Refresh after an accepted status change

Refresh is an ordinary post-save coordinator call, not a broker/timer or a manual second Recalculate button. MON-M0 consumes the shared SH-COORD/MG-M2 access policy and one coherent source snapshot.

Step	Actor	Action	Failure / availability behavior
1	User/UI	Request status change with expectedRevision	Retain entered input if invalid or rejected.
2	F2 + SH-COORD	Authorize; validate; commit atomic source change and next revision once	No success notification/revision advance on denied, conflicting or failed save.
3	SH-COORD / F4	Prove reusable schedule inputs or request F4 recalculation; republish current sourceRevision	If proof/recalculation fails, schedule UNAVAILABLE; safe basic facts still refresh.
4	MON-M0 + F5	Derive current raw counts, progress, known Blocked, assignments and overdue facts	Only unsafe access/coherent source-read failure rejects full dashboard.
5	F6 + UI	Evaluate retained warning types with known inputs; publish one current DashboardSummary	Unsupported warning types unevaluated; do not show stale critical/finish data as current.

Every committed source change advances the one project revision, including status, assignment, notes, labels, roles and dates. Existing-project mutation requests supply expectedRevision. Status-only edits do not change durations/links or shorten the entered plan. Reuse must verify task IDs, graph and durations; any retained values are republished with current sourceRevision and refreshed labels/dates. Otherwise return STALE_RESULT, never an old current-looking result.

If D changes from NOT_STARTED to IN_PROGRESS while C is unfinished, D remains blocked but moves to the raw IN_PROGRESS count. If C completes, D loses the Blocked overlay while keeping its saved status. Opening or filtering the dashboard never writes data. A failed source save preserves the previous saved source and user input; a later schedule-only failure does not roll back a valid source commit.

7 Blocked task design

7 1 Definition and inputs

Input	Owner	Use
Current TaskSnapshot.status	F1/F2	COMPLETED tasks are not unfinished Blocked work.
DependencyResultView	F3	Valid direct links and explicit availability/sourceRevision.
Direct prerequisite status/name	F1/F2 coherent source	Name only the unfinished direct subset.

7 2 Detection rule

```
identifyBlockedTasks(tasks, dependencyView, sourceRevision):
    if tasks is empty:
        return {availability:NO_TASKS, sourceRevision,
                blockedTaskIds:[], byTaskId:{}}
    if dependencyView is NOT_ENABLED or UNAVAILABLE:
        return {availability:dependencyView.availability, sourceRevision,
                blockedTaskIds:null, byTaskId:null,
                reasonCode:dependencyView.reasonCode}
    if dependencyView.sourceRevision != sourceRevision:
        return {availability:UNAVAILABLE, sourceRevision,
                blockedTaskIds:null, byTaskId:null, reasonCode:STALE_RESULT}
    require complete VALID edges with valid same-project endpoints
    for task in tasks where task.status != COMPLETED:
        unfinished = DIRECT prerequisites where status != COMPLETED
        if unfinished is nonempty:
            add task, rawStatus, incomplete prerequisite IDs/names/statuses
    return known BlockedResult at sourceRevision
```

The O(T+E) derivation does not require a schedule result, dates, resource availability or criticality. It does not recurse into prerequisites of prerequisites. F3 validates/saves/deletes links; F5 alone owns this Blocked rule. F5 never edits links or source status.

7 3 Outputs and explanations

Output	Example	Consumer
Known isBlocked	true for D while C incomplete	Raw-status view keeps NOT_STARTED and adds overlay
Direct incomplete prerequisites	C Backend, IN_PROGRESS	Blocked panel/personal view/critical warning
Reason	D Testing has an unfinished prerequisite: C Backend (In Progress). Authorized manual status updates remain allowed.	UI displays the central explanation
Unknown isBlocked	null when F3 not enabled/unavailable	Availability notice, never silent false

```
One blocker: "<Task> has unfinished prerequisite <P> (<raw status>)."
Several: name every incomplete DIRECT prerequisite, once.
Add: "Authorized manual status updates remain allowed."
Completed tasks: no unfinished-blocked explanation.
```

Use names only after authorization inside the project. Unknown/inaccessible project or task references return generic errors without private names, project titles, membership or existence disclosure. In a trusted source view, a dangling/cross-project edge is upstream integrity failure and makes blocker data unavailable, not a fabricated blocker.

7 4 Manual status updates and no lock

A blocked task may receive an authorized manual update under the shared F2 policy. Monitoring has no write/reject authority, force flag or lock. Final completion-before-prerequisite policy is still a team/customer decision; this projection does not invent approval steps.

Saved/source change	Blocked after change	Raw status and note
NOT_STARTED → IN_PROGRESS	Still true if prerequisite unfinished	IN_PROGRESS count; keep Blocked overlay.
IN_PROGRESS → NOT_STARTED	Still true if prerequisite unfinished	NOT_STARTED count; no F5 lock.
Unfinished → COMPLETED if authorized	False for completed task	Enters numerator; final transition policy pending.
Prerequisite completes	Recompute direct incomplete subset	Overlay may clear; dependent raw status retained.

Boundary of responsibility

MG-M2/shared coordination own authorization; F2 owns status saving. F5 explains the condition only. This preserves recoverability and avoids turning a warning into an enforced workflow.

7 5 Worked example

With A/B COMPLETED, C IN_PROGRESS and D NOT_STARTED after B/C, MON-M3 returns D blocked by C only. Raw counts remain2/1/1; completion50%. MON-M6 can warn for D only when current F4 criticalTaskIds includes D. When C completes, D keeps raw status and the overlay clears.

8 Risk monitoring design

8 1 Supported warning conditions

Code	Condition	Required evidence
OVERDUE_TASK	Unfinished task with asOfDate > taskDeadline	Task ID/name, taskDeadline, asOfDate, raw status; candidate date rule.
BLOCKED_CRITICAL_TASK	Known blocked task in current F4 criticalTaskIds	Known direct blockers plus current valid critical set.
PROJECT_DEADLINE_CONFLICT	Current estimatedProjectFinish > projectDeadline	Both project-level dates and current F4 sourceRevision.

8 2 Warning record and evaluation

```
ScheduleWarning = {code, scope:TASK|PROJECT, subjectId,
  subjectName, reason, evidence, sourceRevision}
ScheduleWarningResult = {availability, sourceRevision, items,
  evaluationByType:{overdue, blockedCritical, projectDeadline}}
```

F6 and all FR-15 warning types remain Could Have. MON-M5 supplies the basic overdue list to FR-13 even if F6 is NOT_ENABLED. If F6 is retained, each type reports whether it was evaluated. A safely assembled partial warning result may be VALID while some types are UNAVAILABLE. Empty items never prove no risk for an unevaluated type.

F6 off means NOT_ENABLED with null warning payload. During F4 failure, safe current MON-M5 facts with valid taskDeadline/asOfDate still support the basic FR-13 overdue list. If F6 is enabled, OVERDUE_TASK may also be evaluated independently. BLOCKED_CRITICAL_TASK needs known F5 blockers plus current VALID F4 critical IDs; PROJECT_DEADLINE_CONFLICT needs current finish and projectDeadline. Missing required inputs make those types unevaluated, not no risk; FR-15 remains Could Have.

8 3 Unfinished overdue tasks

Situation	Candidate overdue result	Reason
IN_PROGRESS, taskDeadline20Oct; asOf25Oct	Yes,5 days	Unfinished and25>20.
NOT_STARTED, deadline25Oct; asOf25Oct	No under candidate boundary	Equality is not overdue; confirm full-day/timezone convention.
COMPLETED, past deadline	No	Completed work excluded.
Missing taskDeadline	Not evaluated for that task	Return notEvaluatedTaskIds/reasons; do not assert on-time.
Blocked and past deadline	Yes if known source/date condition	Blocked is unfinished; warning codes may legitimately differ.

Candidate D-04 uses calendar LocalDate values in the project calendar/timezone. Overdue requires unfinished work and known taskDeadline with asOfDate>taskDeadline; equality is not overdue. Missing dates are unevaluated. Chat timezone gives no approval; D-04 remains PENDING.

8 4 Blocked critical tasks

MON-M6 intersects known blockedTaskIds with the full current F4 criticalTaskIds set. It never derives a critical path. A noncritical blocker may appear in the Blocked panel without a critical warning; an unblocked critical task gets no blocked-critical warning. Missing critical IDs produce unknown isCritical, not false. The complete set is certified by F4 NumericPolicy; MON never classifies displayed or rounded float values and defines no local epsilon.

8 5 Project deadline conflict

```
if F6 retained AND schedule.availability == VALID
   AND schedule.sourceRevision == snapshot.revision
   AND estimatedProjectFinish != null AND projectDeadline != null:
    evaluate estimatedProjectFinish > projectDeadline
    emit at most one PROJECT_DEADLINE_CONFLICT if true
else:
    mark projectDeadline evaluation unevaluated with reason
// equality is not a conflict under the candidate convention
// F5/F6 never calculate start + duration as a second finish formula
```

F4 alone supplies estimatedProjectFinish. The recommended D-04 candidate is Day0=plannedStartDate and finish=plannedStartDate+ceil(T) calendar days from canonical unrounded NumericPolicy-resolved T, conditional on explicit scenario adoption or later approval. A 10-day plan against a 9-day window gives a one-day conflict under that candidate. MON never recalculates finish; unknown finish/projectDeadline makes the strict > comparison unevaluated, and equality is not a conflict.

No second schedule algorithm

If F4's calculated finish is unavailable, MON-M7 cannot be evaluated and must be silent except for a neutral availability notice. Computing a local 'start + duration' estimate would create a second, potentially inconsistent schedule result and is therefore prohibited.

8.6 Availability and unsupported states

Upstream state	Basic F5 / overdue	Blocked	Critical / finish warnings	Dashboard treatment
VALID current schedule	Evaluate reliable source facts	Evaluate with current valid F3 links	Evaluate retained F6 type only when all inputs known	One revision-tagged summary
NO_TASKS	Counts0, percentage null	Neutral no-work result	No fabricated critical/finish claim	No-tasks state; wording proposed
F4 NOT_ENABLED / UNAVAILABLE	Keep safe source facts and MON-M5 basic overdue list with valid dates.	Still evaluate if F3 known	With F6 enabled, evaluate OVERDUE_TASK from current MON-M5 facts; critical/finish types are unevaluated.	Neutral analysis notice; dashboard succeeds
STALE_RESULT	Evaluate current source facts	Use current dependency view only	Unknown until verified/recomputed and republished	Do not display old facts as current
F3 off/unavailable	Keep raw statuses/progress/overdue	Unknown; null IDs/flags, not false	Blocked-critical cannot be evaluated; F4 cannot assume empty graph	Scope dependency remains pending
Missing projectDeadline / finish	Keep task overdue facts	As available	Project comparison unevaluated, not no conflict	No invented date; show missing-input notice

8.7 Ordering, duplicates and presentation

Order. Conflicts first, then warnings. Within the same severity, project-level before task-level; task warnings by deadline, then task name.

Duplicates. De-duplicate by (code, subjectId). A task may legitimately appear under two different codes (for example, overdue and blocked-critical); the UI may group them but must not silently drop one.

No pop-ups. Warnings are inline information in the dashboard. There is no modal, toast storm or notification schedule.

No actions taken. The system does not move deadlines, reassign work, change status or mark a project 'at risk' beyond the three explained warnings.

No forecast. The project finish is the calculated schedule result, not a prediction of when the team will actually finish.

8.8 Explicit exclusions

Excluded idea	Why it is excluded
AI delay prediction / risk score	No requirement, training data or validation basis. A score would hide the reason and overstate certainty.
Automatic workload reassignment	Changes project data and authority; outside F5/F6 and explicitly outside the first-release scope.
Frequent or pop-up notifications	Not required; would interrupt users and duplicate the dashboard. Warnings are read in context.
Trend charts / burndown / velocity	No historical status data is specified. Adding history would imply a data model and new requirements.
Critical-path calculation inside F6	Duplicates F4 and risks inconsistent results. F6 consumes the validated critical-task set.
Per-task predicted finish warnings	Not one of the three supported conditions; would require forecasting not defined by the URS.

9 Dashboard UI design

9.1 Goals and constraints

The dashboard is a single project overview for a signed-in project member. It should answer four questions without opening another page: How much is complete? What is the state of each task? What is blocked, and why? What schedule warnings need attention? It is an initial concept, not a final visual design.

One screen, one reading order: summary first, action-oriented detail second, explanations third.

Plain-language labels and explicit counts. Colour supports but never replaces text.

No manual recalculation, no pop-up warnings and no automatic changes to the plan.

Empty, unavailable, access-denied and save-failure states are designed, not left to chance.

9.2 Information architecture

Region	Content	Purpose	Source
Project header	Project name, viewer identity, as-of date, auto-refresh note	Establish context and time	Project snapshot + clock
Overall completion	Percentage, completed/total, task-count basis	Answer 'how far along?'	MON-M2
Task status	Three raw-status counts in a card plus a separate Blocked overlay	Show status composition without hiding the raw state	MON-M1
Schedule health	Counts of conflict, blocked critical and overdue warnings	Give a fast risk summary	MON-M5–MON-M7 / getScheduleWarnings
Assigned to me	Task, status, deadline, blocked- by note, critical flag	Support personal action	MON-M4
Blocked tasks	Task, raw status, direct blocker names/status, explanation	Make FR-09 concrete	MON-M3/MON-M4
Overdue tasks	Unfinished task, deadline, days overdue	Make lateness visible	MON-M5
Schedule warnings	Explained conflict, critical- blocker and overdue items	Connect risk to evidence	MON-M5–MON-M7

9.3 Low-fidelity dashboard wireframe

Figure 2 shows the initial layout for a desktop-width member view. The illustrative project deliberately contains a 4-task plan with 2 completed tasks, one blocked critical task, one overdue task and a one-day project deadline conflict. The wireframe is not a final screen and does not define a component library.

9 3 Low fidelity dashboard wireframe

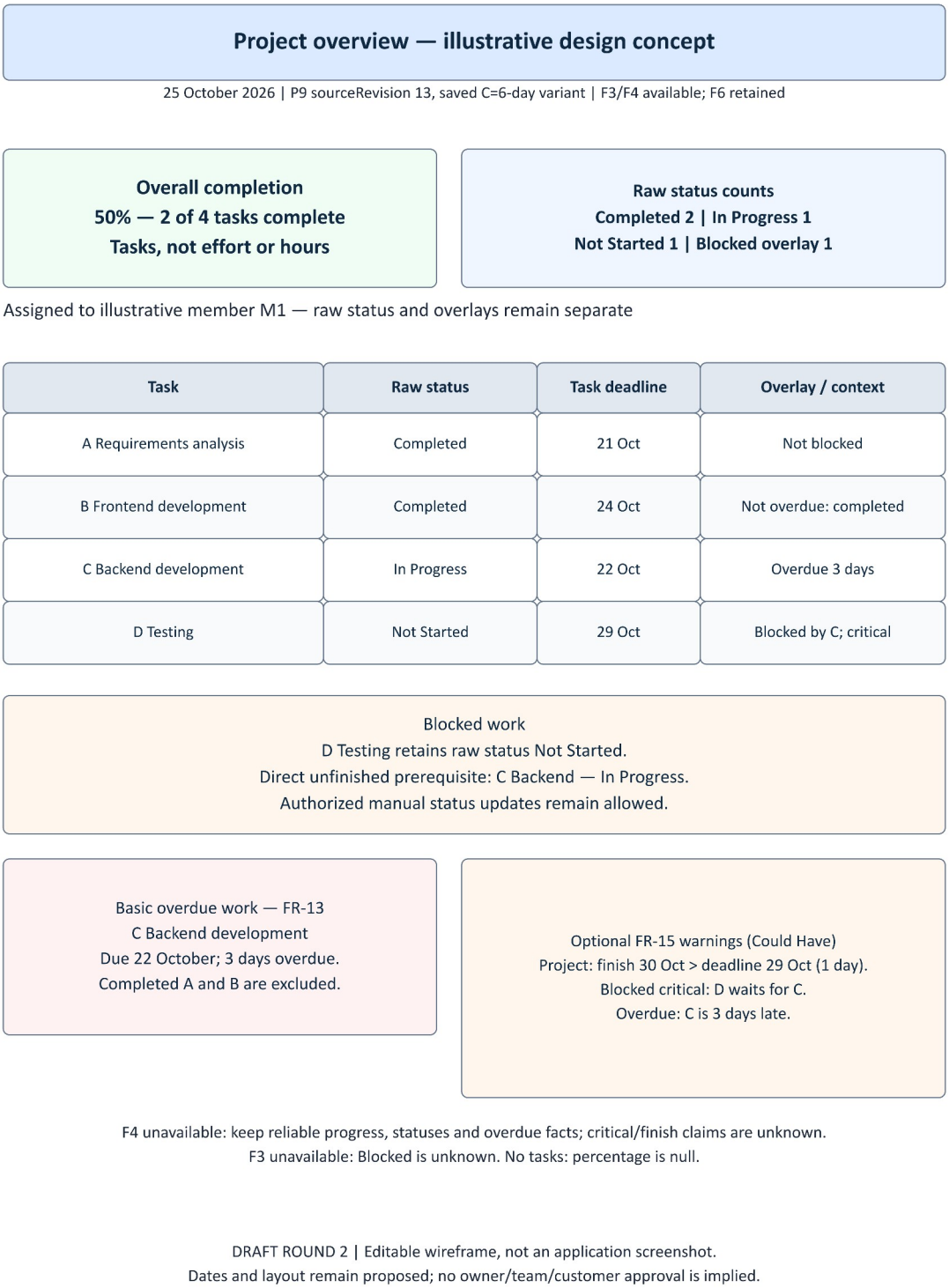


Figure2. Reconstructed editable concept using the revision13 C=6-day fixture. It repairs the original figure's inconsistent completed/overdue labels. Raw status counts and Blocked overlay remain separate; dates are candidate rules, not approved application behavior.

9.4 Section-by-section design

Header and context

The textual header identifies the illustrative project P9, the as-of date, sourceRevision 13 and the saved C=6-day variant. It also names the available F3/F4 capabilities and retained F6 layer. The assignment heading identifies illustrative member M1; the figure does not use a validity dot.

Overall completion

The completion card shows 50% with the explicit '2 of 4 tasks complete' statement and the caption 'Tasks, not effort or hours'. Blocked and unassigned tasks remain in the denominator. For an empty project, the card may use the proposed text 'No tasks yet' with no percentage; final wording remains pending.

Task status

The raw-status card lists Completed 2, In Progress 1 and Not Started 1, with a separate Blocked overlay of 1 task, D. The three raw counts sum to 4; the overlay never subtracts from a raw-status count or adds to the denominator. Personal and blocker views retain each raw status.

Schedule health

The optional F6 layer summarizes only evaluated supported warning types. A known empty warning list may show no warnings only for evaluated types. Missing/stale critical or finish input is explicitly unevaluated; it never means no critical activity or no delay. The basic FR-13 overdue list still appears when F6 is not enabled.

Assigned to me

The table shows the viewer's four tasks with status, deadline, blocked-by/note and critical flags. It is a personal lens on the same snapshot, not a separate project progress calculation. Unassigned tasks are deliberately absent here and still count in the project total.

Blocked tasks

The blocked card answers three questions: which task, which direct prerequisites are unfinished, and what the user should understand. The example shows D Testing waiting for C Backend development (In Progress). The note 'an authorized status update is still allowed' prevents the card from being read as a lock.

Overdue tasks

The overdue card lists only unfinished work with a deadline before the as-of date. It shows the deadline and day difference. A completed task with a past deadline is not listed; the explanatory footnote makes that rule visible.

Schedule warnings

The optional warning panel gives each evaluated item a code, subject and evidence-based reason. With F4 unavailable, only independently evaluable warning types can be assessed if F6 is retained; critical/finish types carry unavailable evaluation status. No pop-ups, forecasting, CPM or automatic reassignment are added.

9.5 Visual encoding and accessibility

Signal	Encoding	Text fallback / rule
Completed	Green fill + 'Completed' label	Never rely on colour alone; label is always present.
In Progress	Blue fill + label	Same.
Blocked	Amber fill + 'Blocked' label + reason	The reason is required; a badge alone is insufficient.
Overdue / conflict	Red text/icon + explicit date or day count	Severity is stated in words; the date is in the sentence.
Focus	Visible outline and logical DOM order	Keyboard path follows summary → tasks → warnings.
Empty state	Plain message, no zero percentage	Text explains that no tasks exist yet.

Figure 2 uses text labels and count cards. The colour mapping in this section is a candidate for later interface refinement, rather than a claim that every encoding appears in this wireframe. Final contrast, keyboard requirements and other accessibility criteria remain pending agreement against the NFR target.

9.6 UI states

State	What the user sees	What must not happen
Normal, tasks present	All cards populated from one DashboardSummary.	No stale fragment from a previous save.
Empty project	Proposed text No tasks yet, zero raw counts and no percentage. Enabled schedule is NO_TASKS; final display wording remains pending.	No 0% completion claim, no division error.
Schedule unavailable	Reliable progress, raw statuses, assignments and overdue facts remain. Blocked depends on F3 availability. Critical/finish claims are unknown.	No invented critical path or project delay warning.
Access denied	Generic Invalid or inaccessible project. No names, counts or membership/existence disclosure.	No partial data leak through counts or error text.
Loading / refresh	Source refresh failure may retain an explicitly stale previous view; schedule-only failure returns current basic facts with availability notices.	Mixing source revisions or flashing contradictory numbers.
Save failed	Inline error at the task edit point; the last valid dashboard remains.	Silently updating the percentage as if the save succeeded.

9.7 Interaction rules

Opening the dashboard or changing a filter never writes data.

A task status save triggers a rebuild; there is no separate calculate button.

The as-of date is visible. In a future version it may be selectable for testing, but that is not assumed here.

Warning text is selectable and readable without hover; tooltips may add detail but cannot carry the only explanation.

The member filter is a view scope, not an authorization bypass. The backend still checks project membership.

9.8 Deliberate omissions

The concept does not include editing, drag-and-drop dependencies, project settings, comments, notifications, trend charts or a risk score. Those are either outside F5/F6 or not supported by the requirements. Keeping them out protects the dashboard's single purpose and avoids implying data the system does not have.

10 Module interfaces

10 1 Contract conventions

All operation names are conceptual, technology-neutral proposals, not implemented APIs or language choices. Operations are read-only. Shared SH-COORD applies MG-M2 membership/permission policy, obtains coherent data and maps public errors before MON-M0 composition. Names never grant access or serve as keys.

Every source snapshot has one revision. Every derived result carries sourceRevision, a reference to that same project counter. Existing-project writes use expectedRevision and advance revision once only after a successful atomic source commit. Rejected/conflicting/save-failed requests publish no successful change or new revision.

An actorId/viewerId is a UserId. Resolve current membership to viewerMemberId before matching TaskSnapshot.assigneeId, which is a project MemberId. A member ID is not a global user ID. asOfDate is explicit. All active date fields distinguish plannedStartDate, taskDeadline, projectDeadline and estimatedProjectFinish.

10 2 Shared data contracts

```
TaskStatus = NOT_STARTED | IN_PROGRESS | COMPLETED
ProjectSnapshot = {projectId, name, plannedStartDate:LocalDate|null,
  projectDeadline:LocalDate|null, members:MembershipRecord[],
  tasks:TaskSnapshot[], dependencyView:DependencyResultView, revision}
MembershipRecord = {memberId, projectId, userId, role}
TaskSnapshot = {taskId, projectId, name, assigneeId:MemberId|null,
  durationDays:number|null, taskDeadline:LocalDate|null, status:TaskStatus}
ProgressNote = {noteId, taskId, authorMemberId, text}
DependencyEdge = {prerequisiteId, dependentId}
DependencyResultView = {availability, sourceRevision,
  edges:DependencyEdge[]|null, reasonCode?}
TaskDisplayView = {taskId, rawStatus, isBlocked:boolean|null,
  isCritical:boolean|null, unfinishedPrerequisites:...|null}
```

F4 analysis requires finite positive durations, including decimals; no integer-only rule is approved. F2 mandatory-field/core-save policy remains D-03/PENDING, and F4 INVALID_INPUT alone cannot reject or roll back a policy-valid F2 save. F2 owns notes and permissions; note policies remain unresolved. F3 off means NOT_ENABLED/FEATURE_NOT_ENABLED with edges=null. Enabled empty tasks mean NO_TASKS/edges=[]; enabled nonempty tasks with complete valid current links mean VALID, including edges=[]. Dependency UNAVAILABLE reasons are DEPENDENCY_INPUT_UNAVAILABLE (missing/unreadable complete input), INVALID_INPUT (invalid graph/records) or STALE_RESULT (coherence unproved). CALCULATION_FAILED belongs to F4, not dependency reads.

```
ScheduleAvailability = VALID | NO_TASKS | NOT_ENABLED | UNAVAILABLE
ScheduleResultView = {availability, sourceRevision, reasonCode?,
  durationDays:number|null, timings:TaskTiming[]|null,
  criticalTaskIds:TaskId[]|null, estimatedProjectFinish:LocalDate|null,
  finishReasonCode?, diagnostics:DurationInputDiagnostic[]|null}
TaskTiming = {taskId, ES, EF, LS, LF, totalFloat}
```

F4 owns these facts. Apply gates in order: off → NOT_ENABLED/FEATURE_NOT_ENABLED; enabled empty tasks → NO_TASKS with null schedule payload; enabled nonempty tasks lacking complete valid same-revision dependencies → UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE. Schedule UNAVAILABLE may instead mean INVALID_INPUT, CALCULATION_FAILED or STALE_RESULT; VALID/NO_TASKS have no failure reasonCode. Missing start/date mapping may leave VALID relative analysis with finish=null. Reuse requires proof and current sourceRevision, never a second counter. Authorized INVALID_INPUT diagnostics follow Section5.5; schedule facts stay null. Failure to certify NumericPolicy timing/criticality gives CALCULATION_FAILED, not a partial critical set. Missing start or unresolved mapping uses MISSING_PLANNED_START/DATE_MAPPING_UNRESOLVED.

10 3 Required operations

Operation	Purpose	Result
calculateProgress	Saved completed/total and empty state	ProgressResult
identifyBlockedTasks	Known direct incomplete prerequisites	BlockedResult
getAssignedTasks	viewerMemberId projection with nullable overlays	AssignedTaskResult
detectOverdueTasks	Source status/deadline + asOfDate facts	OverdueResult
getScheduleWarnings	Optional three-type risk composition	ScheduleWarningResult
buildDashboardSummary	Shared access/snapshot policy plus current view	DashboardResult

10 4 calculateProgress

```
calculateProgress(tasks:TaskSnapshot[], sourceRevision): ProgressResult
ProgressResult = {availability:VALID|NO_TASKS, sourceRevision,
  totalTasks, completedTasks, percentage:number|null}
pre: tasks from one authorized coherent sourceRevision, unique IDs
post: totalTasks=count(tasks); completedTasks=count(status==COMPLETED)
      totalTasks==0 => NO_TASKS, percentage=null
      otherwise percentage=completedTasks/totalTasks*100
```

Blocked is not an input to the formula. A coherent source read has no duplicate IDs; unsafe/conflicting duplicates are SOURCE_UNAVAILABLE before a current summary is published, not silently collapsed by an arbitrary map.

10 5 identifyBlockedTasks

```
identifyBlockedTasks(tasks, dependencyView, sourceRevision): BlockedResult
BlockedResult = {availability, sourceRevision,
  blockedTaskIds:TaskId[]|null, byTaskId:...|null, reasonCode?}
BlockedTask = {taskId, name, rawStatus,
  unfinishedPrerequisites:[{taskId,name,status}], reason}
pre: known complete F3 edges have valid same-project endpoints;
     dependencyView.sourceRevision == sourceRevision after SH-COORD proof
post: COMPLETED tasks absent; known blocked tasks name only incomplete
      DIRECT prerequisites. NO_TASKS => known []/{}; off/unavailable => null.
      Unproved current edges => UNAVAILABLE/STALE_RESULT.
      Other dependency reasons propagate; no F4 CALCULATION_FAILED.
```

F5 computes this rule once and passes the result to MON-M1/MON-M4/MON-M6. It performs neither dependency writes nor cycle validation. F3 is responsible for link validation and atomic maintenance. A corrupt source relationship makes dependent blocker evaluation unavailable; do not expose private IDs or invent blockers.

10 6 getAssignedTasks

```
getAssignedTasks(snapshot, blockedResult, schedule, viewerMemberId):
  {availability, sourceRevision:snapshot.revision, items:AssignedTaskView[]}
AssignedTaskView = {taskId, name, rawStatus, taskDeadline,
  isBlocked:boolean|null, unfinishedPrerequisites:...|null,
  isCritical:boolean|null}
filter: task.assigneeId == viewerMemberId
// viewerMemberId is resolved from authorized viewerId:UserId
// assigneeId is MemberId|null; unassigned tasks remain in totals
```

Use only context tagged with the current snapshot revision; stale overlays become unknown rather than being mixed into current assignments. Use known Blocked/overdue context, then raw status/deadline/name for a proposed UI sort. This is not a new task-priority field or accepted ranking requirement. An unavailable schedule gives isCritical=null, never false masquerading as noncritical.

10 7 detectOverdueTasks

```
detectOverdueTasks(tasks, asOfDate, sourceRevision): OverdueResult
OverdueResult = {availability:VALID|NO_TASKS, sourceRevision, asOfDate, overdueTaskIds,
  items:[{taskId,name,rawStatus,taskDeadline,daysOverdue,reason}],
  notEvaluatedTaskIds, reasons}
for task in tasks where status != COMPLETED:
  missing taskDeadline => not evaluated, not asserted on-time
  else evaluate asOfDate > taskDeadline (candidate rule)
// equality is not overdue under this candidate convention
```

MON-M5 produces basic FR-13 overdue facts independently of F4 and F6. A missing deadline must not be invented. Calendar-day difference is between explicit dates, not duration minus elapsed time. Timezone and deadline-day boundary remain pending.

10 8 getScheduleWarnings

```
getScheduleWarnings(snapshot, schedule, blockedResult, overdueResult, asOfDate):
  ScheduleWarningResult
// f6Enabled is agreed feature configuration supplied by SH-COORD
if not f6Enabled:
  return {availability:NOT_ENABLED, sourceRevision:snapshot.revision,
          items:null, evaluationByType:null, reasonCode:FEATURE_NOT_ENABLED}
evaluate OVERDUE_TASK from current MON-M5 facts/valid dates, independent of F4
if blockedResult is known at snapshot.revision
   and schedule is VALID at snapshot.revision:
  evaluate BLOCKED_CRITICAL_TASK = blocked IDs intersect criticalTaskIds
else mark that type UNAVAILABLE/unevaluated with its applicable input reason
if current estimatedProjectFinish and projectDeadline are known:
  evaluate PROJECT_DEADLINE_CONFLICT using candidate strict comparison
else mark that type UNAVAILABLE/unevaluated with its applicable input reason
return items + evaluationByType + sourceRevision:snapshot.revision + notices
```

A safely composed partial result may be VALID while blockedCritical/projectDeadline entries are UNAVAILABLE. With safe current tasks, valid taskDeadline/asOfDate and MON-M5 facts, F6 may evaluate OVERDUE_TASK despite F4 failure. Independent overdue evaluation does not inherit F4 reasons; schedule-backed types use only their applicable missing-input reason. F6 off keeps FR-15 NOT_ENABLED, while the FR-13 overdue list remains available. Empty items never prove no risk for an unevaluated type. F6 does not calculate CPM or finish; unknown critical/finish fields remain null.

Warning de-duplication remains by(code,subjectId). Different warning codes may legitimately refer to one task. Original ordering proposal is retained: conflicts then warnings, project then task within severity, followed by taskDeadline/name. No pop-up notification, resource optimization or automatic source change is introduced.

Shared public failures include INVALID_INPUT, INVALID_STATUS, INVALID_DEPENDENCY, CIRCULAR_DEPENDENCY, REVISION_CONFLICT, SAVE_FAILED, SOURCE_UNAVAILABLE and unresolved reference-policy outcomes. They never carry successful-change payloads or fabricated current source values.

10 9 buildDashboardSummary

```
buildDashboardSummary(projectId, actorId:UserId, asOfDate): DashboardResult
DashboardResult = {ok:true, summary:DashboardSummary}
                | {ok:false, error:PublicError}
DashboardSummary = {projectId, name, revision, asOfDate,
  progress:ProgressResult, rawStatusCounts, blockedResult,
  assignedTasks, overdueResult, schedule:ScheduleResultView,
  warnings:ScheduleWarningResult, notices}
```

MON-M0 uses SH-COORD and the one MG-M2 permission policy. Reject the whole dashboard only if project/access resolution fails or F1/F2 source data cannot be safely read coherently. Unknown/nonmember/inaccessible projects map to INVALID_OR_INACCESSIBLE_PROJECT with "Invalid or inaccessible project." Task references analogously use INVALID_OR_INACCESSIBLE_TASK. Do not disclose existence, titles, membership or private payloads. PERMISSION_DENIED applies only after an accessible membership context is established.

Schedule-only unavailability/staleness or F3 being off does not reject current safe base facts. Return reliable raw statuses, completion, assignments and basic overdue facts; return Blocked only when dependency input is known. Suppress unsupported critical/finish warnings with capability notices. Do not expose schedule-only STALE_RESULT as a top-level fatal dashboard error.

10 10 What monitoring needs from F1 and F2

Data	Purpose	Owner / rule
Project ID/name/plannedStartDate/projectDeadline/revision	Identity and date context	F1 source; no generic deadline/version alias
MembershipRecord with memberId/userId/role	Access and personal member scope	MG-M2 policy via SH-COORD
Task IDs/name/assigneeId/status/taskDeadline/durationDays	Progress, blocker lookup, assigned and overdue views	F2 source; unique IDs and three TaskStatus values
Progress notes if presented	Readable partial-work context	F2; no change to percentage from a note
SavedChange after successful source commit	Rebuild at returned revision	No success event on rejection/failure; no broker required

10 11 What monitoring needs from F3 and F4

F3 supplies complete valid direct edges at DependencyResultView.sourceRevision = snapshot.revision. SH-COORD reauthorizes and re-reads or proves edge/task-universe consistency before current publication, including after unrelated notes/roles change (Section 5.4). Unproved views are unavailable, never relabeled current. F4 supplies ScheduleResultView and nullable critical/timing/finish facts; enabled nonempty analysis without trusted F3 input uses UNAVAILABLE/DEPENDENCY_INPUT_UNAVAILABLE. F5 never writes F3 records; F6 never runs CPM.

Lifecycle source coordination: departing-member tasks remain in project totals with assigneeId=null; statuses, durations, notes and links remain. Reauthorize membership and rebuild personal views. Task deletion must consult F3/reference policy; unresolved referenced deletion returns REFERENCE_POLICY_UNRESOLVED. F5 never edits links, cancels tasks, creates a fourth status or automatically reassigns work.

11 Validation scenarios

11 1 Status and approach

All ten original scenarios are retained below as proposed cases, NOT RUN. Document reconstruction, static checks and arithmetic verification do not execute an application. Later evidence must record actual output, environment, revision, executor and artifacts.

11 2 Common scenario data

Task	Duration	Prerequisites	Saved status	taskDeadline
A Requirements	2	None	COMPLETED	21 Oct2026
B Frontend	3	A	COMPLETED	24 Oct2026
C Backend	4	A	IN_PROGRESS	22 Oct2026
D Testing	2	B,C	NOT_STARTED	29 Oct2026

Shared illustrative fixture: P9 revision12, plannedStartDate20Oct2026, projectDeadline29Oct2026, asOfDate25Oct2026. A/B completed gives50%; raw counts2/1/1; D blocked by C; C overdue3days. Base duration8days, A/C/D critical, B float1, estimatedProjectFinish28Oct (no project conflict). Saving C=6 yields revision13, duration10days, finish30Oct and1-day project conflict; B float3. Date values are conditional D-04 candidates using ceil(T) on canonical unrounded resolved duration; no approved mapping is inferred. Figures use the saved C=6 variant and illustrative memberM1 for all assignments, not real team work allocation.

Original independent TC-05 and TC-09 fixtures remain independent modifications of the base setup. They are not falsely combined into one current snapshot. TC-09 keeps its9Nov/8Nov comparison example separately.

ID	Scenario	Given	When	Expected result
TC-01	Completion percentage calculation	4 tasks; A and B COMPLETED; C IN_PROGRESS; D NOT_STARTED and blocked by C.	Call calculateProgress and MON-M1.	Completed2, total4, percentage50%; raw counts COMPLETED2, IN_PROGRESS1, NOT_STARTED1; blocked overlay1. Sum only raw counts=4.
TC-02	Empty project handling	Project with 0 tasks; no dependencies.	Build the dashboard.	Progress NO_TASKS, total0, percentage null; enabled schedule NO_TASKS with null duration/critical/finish. No division or fabricated0%/0-day delivery.
TC-03	Correct blocked-task detection	A and B COMPLETED; C IN_PROGRESS; D depends on B and C.	Call identifyBlockedTasks and MON-M1.	D raw NOT_STARTED remains counted there; isBlocked true and direct unfinished prerequisite C named. Completing C clears the overlay.
TC-04	Blocked task with manual status update	D is blocked by C; an authorized user saves D as IN_PROGRESS.	Persist the status, rebuild the dashboard.	D raw IN_PROGRESS is retained/counts there; Blocked remains while C unfinished. Authorized update is not locked by F5; no duration or graph change.
TC-05	Overdue unfinished tasks	As-of 25 Oct. B IN_PROGRESS due 20 Oct; C IN_PROGRESS due 22 Oct; A COMPLETED due 10 Oct; D blocked due 28 Oct.	Call detectOverdueTasks.	B and C are overdue with 5 and 3 days; D is not yet overdue; Completed A is excluded. Warnings name the task, deadline and reason.
TC-06	Unavailable schedule calculation	Valid tasks and dates; ScheduleResultView availability = UNAVAILABLE.	Build the dashboard.	Dashboard succeeds with reliable progress/status/assignment/overdue facts and known Blocked data. Unsupported critical/finish facts null and warning types unevaluated; neutral notice.
TC-07	Non-project-member access denial	Viewer ID is not in the project's member list.	Call buildDashboardSummary.	Public INVALID_OR_INACCESSIBLE_PROJECT; generic message identical for unknown/nonmember/inaccessible project. No private payload or existence inference.
TC-08	Dashboard refresh after a saved status change	Base data: C IN_PROGRESS, D blocked, completion 50%. User saves C as COMPLETED.	Persist, rebuild, compare sourceRevision with snapshot revision.	Completion75%; D overlay clears and raw status retained. Relative timings reused only after input proof, then republished at new sourceRevision; otherwise unavailable.
TC-09	Project deadline conflict	Calculated project finish 9 Nov 2026; project deadline 8 Nov 2026; schedule valid.	Call getScheduleWarnings.	Current VALID finish9Nov > projectDeadline8Nov gives one1-day conflict. Equality gives no conflict for this evaluated case. Unavailable schedule makes this comparison unevaluated; hide the warning, never assert no conflict.
TC-10	Critical-blocker warning and non-critical block	D is blocked by C and D is in criticalTaskIds. E is blocked but not critical. Schedule valid.	Call getScheduleWarnings.	One blocked-critical-task warning for D naming C. No critical warning for E. Both may still appear in the blocked card. With schedule UNAVAILABLE, neither critical warning appears.

11.3 Additional checks to add later

Boundary: deadline exactly equal to as-of date; leap day; month/year boundary; project time zone.

Multiple blockers: all names returned, no duplicate names, stable order.

Parallel critical activities: every critical ID is considered; no single-path assumption.

Conflicting duplicate task IDs: reject an incoherent source read; never silently choose a record or fabricate counters.

Large fixture: counts and warnings remain linear in tasks + edges; target threshold still to be agreed.

Completed task with unfinished prerequisite: displayed Completed and excluded from blocked; record the product decision.

11.4 Coverage map

Requirement / rule	Covered by
FR-09 blocked tasks and reasons	TC-03, TC-04, TC-10
FR-13 basic Dashboard	TC-01, TC-02, TC-05 basic facts, TC-06, TC-07, TC-08; Blocked acceptance remains D-01.
FR-14 completion status	TC-01, TC-02, TC-04, TC-08
FR-15 schedule warnings	TC-05, TC-06, TC-09, TC-10
Access / project isolation (NFR-04)	TC-07
Refresh and warning consistency (FR-04, DR-04, NFR-05)	TC-08

12 Outstanding decisions

The design is deliberately explicit about what is not settled. These items should be closed with the team and checked against the URS before implementation; each has a concrete impact on the module contracts.

#	Question / decision needed	Impact	Suggested owner / timing
Q1	Revised URS/course/analysis now located and checked. Formal artifact identity is resolved; owner/team/customer approval and acceptance criteria are not.	Do not keep obsolete missing-source or wrong-ID assumptions.	F5/F6 owner with team; before design baseline.
Q2	Date convention: is the deadline inclusive, what time zone applies, and how does F4 map duration to a calculated finish date?	Overdue boundary, project conflict day count and expected test values.	F3/F4 + team; next design review.
Q3	Who may update a task status (assignee only, project leader, any member), and what happens to authorization on a blocked task?	F2 boundary and TC-04/TC-07; F5/F6 remains read-only.	F1/F2 + team; next design review.
Q4	Percentage presentation: one decimal, whole percent, or exact fraction? Confirm rounding and trailing-zero rules.	DisplayText and acceptance expectations such as 1/3.	F5 owner + team; next design review.
Q5	Retain departing members' tasks as unassigned per FR-02. Confirm only the presentation of unassigned work; no automatic reassignment or cancellation.	Project totals retain these tasks; personal views use current membership/assigneeId.	F1/F2 + F5; next design review.
Q6	How is a completed task with unfinished prerequisites treated in the dependency/status consistency rule?	Blocked precedence, warning policy and data integrity.	Team; before status-update implementation.
Q7	Apply the proposed one-revision protocol; confirm transaction/reuse mechanism with team before implementation.	MON-M0/MON-M6/MON-M7 sourceRevision gate and refresh sequence.	F3/F4 + application layer; design review.

#	Question / decision needed	Impact	Suggested owner / timing
Q8	A missing projectDeadline or plannedStartDate suppresses the unevaluable project comparison only; confirm final date UX.	Availability UX; no invented dates.	F5 owner + F3/F4; next design review.
Q9	What are the representative task/dependency counts and acceptable dashboard refresh time?	NFR verification and performance test design.	Team; next design review interview/threshold decision.
Q10	How should dependency deletion and invalid status combinations be surfaced without creating a new warning type?	Integrity handling and F2 error messages.	F3/F4 + F1/F2; before integration.

13 Summary

The design retains seven small business modules and a thin MON-M0 facade. F5 counts three saved statuses, computes completed/total, derives direct Blocked overlays, builds personal views and basic overdue facts. F6 optionally composes unfinished-overdue, blocked-critical and project-deadline warnings, without a second scheduling algorithm. MON-M5 facts are reusable; their simplicity never changes FR-15's Could priority.

The dashboard retains its reading order and explanatory approach.50% means2of4 saved task records. Blocked is an independent overlay, not a fifth task or a fourth counting category. Empty, denied-access, save-failure, F3-off, F4-off and unavailable/stale analyses are explicit states; only unsafe source/access failure rejects the whole dashboard.

This ROUND2 proposal is ready for owner/team review, not approved coding sign-off. The revised URS artifact and formal identifier meanings are now verified. Date/transition/permission/scope/acceptance decisions remain open. Finalize detailed interfaces after agreement, then execute the proposed scenarios against a real implementation and save actual evidence separately.

Appendix A Traceability matrix

Requirement / rule	Design element	Evidence in this document	Proposed validation
FR-09	MON-M3, MON-M1, MON-M4, MON-M0	Sections 5, 7, 9, 10	TC-03, TC-04, TC-10; IT25/27 (NOT RUN)
FR-13 basic Dashboard	MON-M0 / MON-M1 / MON-M2 / MON-M4 / MON-M5 + UI	Sections 2.5, 4, 6, 9, 10	TC-01, TC-02, TC-05 basic facts, TC-06, TC-07, TC-08; IT25/27 (NOT RUN)
FR-13 Blocked subview	MON-M3; dependent on Should Have FR-09 / F3	Sections 2.4/2.5, 7, 10; release decision D-01 pending	TC-03, TC-04, TC-06; full blocker acceptance pending D-01; IT25/27 (NOT RUN)
FR-14	MON-M1, MON-M2	Sections 5, 6, 10	TC-01, TC-02, TC-04, TC-08; IT25 (NOT RUN)
FR-15	MON-M5, MON-M6, MON-M7 + getScheduleWarnings	Sections 8, 9, 10	TC-05, TC-06, TC-09, TC-10; IT25/27 (NOT RUN)
FR-04	F2 authorized status-save → refreshed F5 read model	Sections6.5,10	TC-04,TC-08
FR-05	F2 short progress-note source; no percentage change from notes	Section10 upstream data	F1/F2 note-save design case; not executed
NFR-01	One-screen hierarchy, plain language	Sections 9.1–9.8	UI review; walkthrough
NFR-02	Browser and Device Support	Sections2.3,9	Agreed Chrome/Edge/device checks later
NFR-03	O(T+E) read model; one summary per save	Sections 3.3, 6.2, 10	Large fixture later
NFR-04	Membership check before composition	Sections 3.3, 10.9	TC-07; IT27 (NOT RUN)
NFR-05	Reliability: error/input/data preservation and recovery	Sections6.5,9.6,10	Proposed save/reject/reopen/recovery checks; IT27 (NOT RUN)
DR-04	Blocked warning, underlying raw status and authorized manual updates	Sections5,7,10	TC-03,TC-04,TC-08; IT25/27 (NOT RUN)
BR-07/08	Three saved statuses; independent Blocked overlay; denominator counts task once	Sections 5, 6, 7	TC-01–TC-04
BR-09/10	Three warning conditions; no forecast	Section 8	TC-05, TC-06, TC-09, TC-10

Appendix B Glossary

Term	Meaning in this document
Blocked	A derived condition for an unfinished task with at least one direct prerequisite that is not Completed. Not a saved status.
Completion percentage	Completed distinct tasks divided by total distinct tasks, multiplied by 100. Task-count based, not effort based.
Critical activity	A task with mathematical zero total float certified by F4 NumericPolicy in a valid current schedule. The complete set is owned by F4; MON performs no display-rounded or unqualified binary-zero classification.
TaskDisplayView	View containing rawStatus plus nullable isBlocked and isCritical; no fourth stored status or exclusive counting category.
DR	Formal domain requirement; DR-04 is Blocked work as a warning allowing authorized updates.
F5 / F6	Progress Tracking and Schedule Risk Monitoring, the two functional areas owned by Zhang Yuyang.
LocalDate	A calendar date without a time-of-day component, interpreted in the project's agreed time zone.
MON-M0–MON-M7	MON-M0 is the thin facade; MON-M1..5 provide F5 facts; MON-M6/7 and the composer provide optional F6 warnings.
NFR	Formal non-functional requirement. NFR-01 Usability;02 Browser and Device Support;03 Performance;04 Security and Access Control;05 Reliability.
ScheduleResultView	Read-only F4 facts with four-state availability, nullable schedule payload and sourceRevision. F5/F6 never recalculate CPM.
Project revision	One project-local counter advanced once by each successfully committed atomic source change. sourceRevision refers to this same counter.
Warning	An explained, inline risk item with a code, subject, reason and evidence. It is not a pop-up and does not change data.

Round2 source and preservation record

Primary source:
00_Source_Materials/Task2_URS/Student_Project_Planning_and_Progress_Tracking_System_URS_REVISED.pdf,24pages. NFR01/02 p17; NFR03/04/05 p18; FR09 p13; FR13/14 p15; FR15 p16; DR04 p19; priorities pp5,20. The recovered revised artifact is byte-identical to the earlier24-page attachment; this does not prove customer approval.

Other sources: Chapter 6 lecture105pages; revised requirements analysis36pages; actual Proposal(1)8pages. The original31-page monitoring PDF is preserved at Individual_Designs/ZhangYuyang/Sources/. Its SHA-256 is b163d3a39cdde757a100ce3a4586d36f214d63b514918e5b9c83c1871025cf10.

All original section purposes and ten scenario identifiers remain represented. Conflicting priority/ID/count/version/error statements are replaced at their active use sites. Date boundaries, status transitions, presentation precision, FR13/F3 scope dependency and approval remain open. New figures are native editable sources, not embedded source screenshots. This review makes no claim that Zhang Yuyang approved the editorial revisions. The current shared integration plan is IT01-IT27:27 NOT RUN cases. IT25 passes certified relative CPM facts to MON-M3/M6; IT26 checks upstream NumericPolicy classification; IT27 checks authorized invalid-duration diagnostics and safe F5/known Blocked retention. The original ten TC cases remain NOT RUN, not runtime results.